baseline_regression.ipynb

All Cups 3521: baseline регрессии цен

Цель — пройти полный путь от контракта задачи до валидного `baseline_submission.csv`. Практика — 35 минут: 25 минут на рабочий baseline и 10 минут на одно изменение и обсуждение. Baseline сравнивает константный прогноз и Ridge на одном временном holdout. Метрика контеста — MAE, поэтому случайное разбиение здесь может дать слишком оптимистичную оценку.

Что проверяем до модели

- Объект: объявление о продаже недвижимости.
- Target: `price`.
- Момент решения: после появления объявления, без будущих данных.
- Локальная проверка: последние по времени объявления.
- Контракт отправки: ровно `id,price`, порядок ID как в sample submission.

```
from pathlib import Path
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.metrics import (
    mean_absolute_error,
    mean_absolute_percentage_error,
    mean_squared_error,
    r2_score,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

def locate_data_dir():
    candidates = [
        Path("data"),
        Path("competition/data"),
        Path("lesson_04_supervised_learning/competition/data"),
    ]
    for candidate in candidates:
        if (candidate / "train.csv").exists():
            return candidate
    raise FileNotFoundError("Run the notebook from competition/ or the repository root")

DATA_DIR = locate_data_dir()
print("Data directory:", DATA_DIR)
```

1. Загружаем и проверяем данные

`id` — идентификатор, а не числовой сигнал. `date` превращаем только в признаки, доступные в момент публикации объявления. Пропуски обрабатываем внутри Pipeline.

```
train = pd.read_csv(DATA_DIR / "train.csv", parse_dates=["date"])
test = pd.read_csv(DATA_DIR / "test.csv", parse_dates=["date"])
sample = pd.read_csv(DATA_DIR / "sample_submission.csv")

print("train:", train.shape, "test:", test.shape, "sample:", sample.shape)
print("target:", [column for column in train.columns if column not in test.columns])
print("dates:", train["date"].min(), "→", train["date"].max())
print(train["price"].describe(percentiles=[0.01, 0.5, 0.99]).round(0))
print("Top missing shares:")
print(train.isna().mean().sort_values(ascending=False).head(5).round(3))
```

2. Строим признаки без утечки

Категориальные коды нельзя трактовать как физическую шкалу. Для них используем one-hot encoding. Числовые признаки заполняем медианой и масштабируем. Все обучаемые преобразования находятся внутри Pipeline.

```
DATE_ORIGIN_YEAR = 2011

def make_features(frame):
    result = frame.drop(columns=["id", "price"], errors="ignore").copy()
    result["year"] = result["date"].dt.year
    result["month"] = result["date"].dt.month
    result["month_index"] = (result["year"] - DATE_ORIGIN_YEAR) * 12 + result["month"] - 1
    return result.drop(columns="date")

ordered = train.sort_values(["date", "id"]).reset_index(drop=True)
X = make_features(ordered)
y = ordered["price"]
X_test = make_features(test)

categorical_features = ["street_id", "build_tech", "balcon", "g_lift"]
numeric_features = [column for column in X.columns if column not in categorical_features]

def make_preprocessor():
    numeric = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])
    categorical = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("one_hot", OneHotEncoder(handle_unknown="ignore")),
    ])
    return ColumnTransformer([
        ("numeric", numeric, numeric_features),
        ("categorical", categorical, categorical_features),
    ])

split_index = int(len(ordered) * 0.80)
X_train, X_valid = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_valid = y.iloc[:split_index], y.iloc[split_index:]
print("Validation starts:", ordered.loc[split_index, "date"].date())
print("train rows:", len(X_train), "validation rows:", len(X_valid))
```

3. Оцениваем все модели одинаково

MAE — целевая метрика контеста. Остальные метрики помогают понять характер ошибки: RMSE сильнее штрафует крупные промахи, R2 сравнивает с константой, MAPE измеряет относительную ошибку, но нестабилен около нуля.

Для объекта i обозначим остаток $e_i = y_i - \hat{y}_i$. Метрики на одном holdout:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2}.$$

MAE и RMSE имеют единицы цены. Ridge обучается с квадратичной потерей и L2-штрафом, но сравниваем модели по validation MAE. R^2 может быть отрицательным; MAPE нестабильна при target около нуля и в scikit-learn возвращается в долях.

```
def regression_metrics(y_true, prediction):
    prediction = np.maximum(np.asarray(prediction), 0)
    mse = mean_squared_error(y_true, prediction)
    return {
        "MSE": mse,
        "MAE": mean_absolute_error(y_true, prediction),
        "RMSE": np.sqrt(mse),
        "R2": r2_score(y_true, prediction),
        "MAPE": mean_absolute_percentage_error(y_true, prediction),
    }

models = {
    "Dummy median": DummyRegressor(strategy="median"),
    "Ridge alpha=1": Pipeline([
        ("preprocessing", make_preprocessor()),
        ("model", Ridge(alpha=1.0, solver="lsqr")),
    ]),
}

rows = []
fitted_models = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    prediction = model.predict(X_valid)
    rows.append({"model": name, **regression_metrics(y_valid, prediction)})
    fitted_models[name] = model

metrics = pd.DataFrame(rows).set_index("model")
print(metrics.round({"MSE": 0, "MAE": 0, "RMSE": 0, "R2": 3, "MAPE": 3}))
```

4. Интерпретируем baseline

Dummy с медианой — простой ориентир для MAE. Сравнение с Ridge показывает, дают ли признаки и модель пользу на будущих объявлениях. Выбор модели делаем по validation MAE, а не по train score.

```
ridge_model = fitted_models["Ridge alpha=1"]
valid_prediction = np.maximum(ridge_model.predict(X_valid), 0)
errors = pd.DataFrame({
    "actual": y_valid.to_numpy(),
    "prediction": valid_prediction,
})
errors["absolute_error"] = (errors["actual"] - errors["prediction"]).abs()
print("Largest validation errors:")
print(errors.nlargest(10, "absolute_error").round(0).to_string(index=False))
```

5. Обучаем Ridge на всех данных и собираем отправку

Формат проверяем локально до загрузки. Отрицательные цены обрезаем до нуля, хотя правильнее исследовать причину таких прогнозов и распределение target.

```
final_model = Pipeline([
    ("preprocessing", make_preprocessor()),
    ("model", Ridge(alpha=1.0, solver="lsqr")),
])
final_model.fit(X, y)
test_prediction = np.maximum(final_model.predict(X_test), 0)

assert test["id"].notna().all() and test["id"].is_unique
assert sample["id"].notna().all() and sample["id"].is_unique
assert set(test["id"]) == set(sample["id"]), "Test and sample IDs must match"
prediction_by_id = pd.Series(test_prediction, index=test["id"], name="price")
submission = sample[["id"]].copy()
submission["price"] = submission["id"].map(prediction_by_id)
submission.to_csv("baseline_submission.csv", index=False)

assert submission.columns.tolist() == ["id", "price"]
assert submission["id"].equals(sample["id"])
assert len(submission) == len(test)
assert np.isfinite(submission["price"]).all()
assert (submission["price"] >= 0).all()
print("baseline_submission.csv ready:", submission.shape)
print(submission.head())
```

Самостоятельный эксперимент: последние 10 минут практики

0–2 минуты: сохраните исходный `baseline_submission.csv` отдельной копией, запишите исходную метрику и сформулируйте гипотезу.

2–7 минут: добавьте новую кодовую ячейку после baseline и выполните пример ниже. Меняется один параметр. Исходная модель, представление, split/подвыборка и CSV сохраняются. Улучшение не гарантировано: отсутствие заметного эффекта тоже результат.

```
from sklearn.base import clone

# Исходную модель и baseline_submission.csv не меняем.
experiment = clone(models["Ridge alpha=1"]).set_params(model__alpha=10.0)
experiment.fit(X_train, y_train)
experiment_mae = regression_metrics(y_valid, experiment.predict(X_valid))["MAE"]
print("MAE alpha=1:", metrics.loc["Ridge alpha=1", "MAE"])
print("MAE alpha=10:", experiment_mae)
```

7–10 минут: сравните числа и объясните наблюдение двумя предложениями. Что изменилось в модели и чего этот эксперимент ещё не доказывает? Не выбирайте ответ по скрытому leaderboard вместо локального протокола.

Полиномиальные признаки, Lasso и робастные потери остаются дополнительными экспериментами.